Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет информационных технологий и управления

Кафедра вычислительных методов и программирования

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему:

«Система организации электронных кулинарных рецептов»

Выполнил: гр. 421703 Рассохов Е. П.

Проверила: Новицкая Л. И.

Минск 2025

РЕФЕРАТ

В современном мире кулинария играет важную роль в повседневной жизни человека. Для более комфортного управления рецептами, без которых, возможно не существовало бы этой сферы, было разраотано это приложение, которое позволяет производить разного рода операции, благодоря которым работа с цулинарной книгой станет немного проще и интереснее.

Цель программы — создание удобного хранинилища рецептов, которое позволяет:

- Добавлять и редактировать рецепты;
- Искать блюда по названию, типу, ингредиентам и калорийности;
- Сортировать рецепты по заданным параметрам;
- Сохранять данные для дальнейшего использования.

Данная записка состоит из 5 основных пунктов:

- Структуры и файлы;
- Алгоритмы сортировки;
- Алгоритмы поиска;
- Пользовательские функции
- Работа программы.

Работа представляет ценность как для обычного пользователя, которому будет гораздо проще распоряжаться рецептами, так и для заведений, которые специализируются на сфере питания.

СОДЕРЖАНИЕ

Введение 6
1 Структуры и файлы 7
1.1 Описание структуры Recipe 7
1.2 Работа с файлами 8
2 Алгоритмы сортировки 11
2.1 Сортировка пузырьком 11
2.2 Сортировка по категориям 12
2.3 Быстрая сортировка 13
2.4 Сортировка выбором 14
3 Алгоритмы поиска 16
3.1 Линейный поиск 16
3.2 Интерполяционный поиск 17
4 Пользовательские функции 19
5 Описание работы программы 23
Заключение 26
Список использованных источников 27
Приложение А(обязательное)Листинг кода 28
Приложение Б(обязательное)Блок-схема работы программы 41

ВВЕДЕНИЕ

Человек, независимо от сферы использования, стремится систематизировать свои знания для более простого доступа к информации. Для решения этой проблемы в данном документе описан работа на тему «Система организации электронных кулинарных рецептов».

Благодаря этому приложению пользователь будет иметь возможность гибко изменять список рецептов, включая добавление, удаление и редактирование каждого элемента по отдельности. Для этой цели будет использоваться язык программирования C++, который имеет возможность реализовать вышеперечисленные функции.

В этом приложении реализована работа с бинарными и текстовыми файлами, позволяя хранить информацию и получать к ней быстрый доступ; записывать все совершенные действий в лог файл для доступа к истории изменений; управление содержимым файлов для внесения поправок в список рецептов. Также для удобства реализаций алгоритмов поиска и сортировки, все рецепты из файла будут записаны в массив структур.

Целями работы являются:

- Создание блок-схемы работы программы;
- Представление структур для систематизации электронных рецептов;
- Разработка функций для удаления, создания и редактирование данных в файлах, в которых будет храниться информация о рецептах;
- Взаимодействие с лог файлом для записи всех операций, проведённых над файлом с записями о рецептах;
- Создание функций для сортировки и поиска рецептов по определённым критериям;
- Запись рецептов в массив структур.

В итоге реализации этой работы будет создано приложение для систематизации электронных рецептов, позволяющее гибко вносить изменения в уже созданные списки рецептов или создавать новые списки рецептов, что позволит работать с рецептами ещё удобнее.

1 СТРУКТУРЫ И ФАЙЛЫ

1.1 Описание структуры Recipe

Структура представляет собой составной тип данных, который состоит из других компонентов. При этом в отличие от массива эти компоненты могут представлять различные типы данных [1].

Нередко структуры применительно к C++ также называют классами. И в реальности различия между ними не такие большие. Структура также может определять переменные, функции, конструкторы, деструкторы. Однако обычно структуры служат для хранения каких-то общедоступных данных в виде публичных переменных. Для остальных сценариев используются классы [1].

В этой работе содержится структура `Recipe`, которая выглядит так:

```
struct Recipe{
  char type[18];
  int typenum;
  char name[100];
  int kall;
  int time;
  char description[200];
};
```

Каждому полю соответствует определенная характеристика:

- тип блюда (название);
- тип блюда (номер);
- название блюда;
- калорийность блюда;
- время приготовления блюда;
- описание рецепта.

Поля `type`, `name` и `description` являются масивами типа `char` используеися для удобного вывода и работы со строками. Поле `typenum` целочисленного типа `int` — для удобства реализации алгоритма для вывода блюд по категориям. Поле `kall` целочисленного типа `int` — для хранения калорийности рецепта. Поле `timt` целочисленного типа `int` — для хранения времени приготовления рецепта. Поле `description` является массивом типа `char` — для хранения описания рецепта.

Для того, чтобы хранить несколько рецептов и иметь возможность сортировать их и осуществлять поиск, в дальнейшем рецепты будут записаны в массив структур.

1.2 Работа с файлами

Файл — именованная область данных на носителе информации, используемая как базовый объект взаимодействия с данными в операционных системах [2]. Двоичный файл — последовательность произвольных байтов. Название связано с тем, что байты состоят из бит, то есть двоичных цифр [2].

Для того чтобы не допустить потерю данных и записи истории использования функций, эта работа содержит возможности сохранения данных в файлы для дальнейшей работы. В коде содержаться встроенные функции для работы с файлами такие как:

- `fread();`
- `fseek();`
- `fwrite();`
- `fprintf();`
- `ftell();`
- `fopen();`
- `fclose()`.

Функция `fread()` используется в программе для записи в структуру данных из файла. Для её работы ей передаётся ссылка на экземпляр структуры, в которую будет записана информация, размер структуры, которая будет записана, количество записываемых структур и ссылка на файл, из которого будет считана информация. Для расчета размера структуры, используется функция `sizeof()`: `fread(&dishes, sizeof(Recipe), 1, file)`.

Функкция `fseek()` используется в работае для передвижения курсора по бинарному файлу для подсчёта количества структур (рецептов) в файле. Для её использования ей передаются ссылка на файл, количество байт для смещения и место для смещения:

- `SEEK_SET` начало файла;
- `SEEK_CURR` настоящее положение;
- `SEEK_END` конец файла.

В коде реализация выглядит так: `fseek(file, 0, SEEK_SET)`.

При записи в файл данных используется функция `fwrite()` в программе. Для её работы в работае ей передаётся ссылка на экземпляр структуры, которая будет записана, размер структуры, которая будет записана, количество записываемых структур и ссылка на файл, в который будет записана информация: `fwrite(&dishes, sizeof(Recipe), 1, file)`.

Функция fprintf() используется для форматированного вывода в файле. Ей передаётся ссылка на файл, строка форматирования и модификаторы: `fprintf(file, "%-15s%-20s%-20s%-20s%-100s\n", "Name:", "type", "time", "calories", "description")`. В работае в основном эта функция используется для форматированного вывода в лог файл и консоль, при этом для вывода в консоль используется поток вывода `stdout;`

Функция `ftell()` возвращает количество байт от начала файла до курсора. Ей передаётся ссылка на файл: `ftell(file)`.

Эта функция необходима для подсчета количества структур, так как при помощи команд `fseek()` и `sizeof()` мы получаем количество байт от начала файла до конца и полученное число делим на размер структуры. Для удобства эти команды помещены в функцию structCount:

```
int countStructs(FILE *file){
  fseek(file, 0, SEEK_END);
  int num = (int)(ftell(file)/sizeof(Recipe));
  fseek(file, 0, SEEK_SET);
  return num;
}
```

Функция `fopen` используется при открытия файла для дальнейшей работы с ним. Для её работы необходимо передать путь к файлу и 1 из 6 режимов открытия, а именно:

- `r` используется для чтения информации из файла;
- `a` используется для добавления новой информации без изменения старых данных;
- `w` используется для полной перезаписи данных.

Остальные 3 режима, а именно `wb`, `rb` и `ab` выполняют те же роли что и их аналоги, только с тем отличием, что они используются для работы с бинарными файлами.

Для корректной работы с файлами после ненадобности, файл нужно закрыть при помощи команды `fclose()`, которая принимает ссылку на закрываемый файл: `fclose(file)`.

Чтобы не вызвать ошибки при чтении файла, вызывается функция `IsExistsFile()`, принимающая как аргументы ссылку на файл и имя самого файла в форме массива символов:

```
int IsExistsFile(FILE* file, char fileName[100]){
  file = fopen(fileName, "rb");
  int check = !file? 1: 0;
  fclose(file);
  return check;
}
```

При помощи функции `create()` создаётся новый файл для дальнейшей работы с ним. Сама функция принимает как аргументы ссылку на файл и имя файла как массив символов и выглядит так:

```
void create(FILE *file, char fileName[100]){
  file = fopen(fileName, "wb");
  if(!file){
    cout << "Create file error!\n";
    return;}
```

```
  fclose(file);}
```

Функция `structRead()` используется для считывания структур из бинарного файла и записи в массив для дальнейшего использования в программе. Для её работы используются такие аргументы как ссылка на файл, массив, состоящий из структур и количество считываемых структур. В коде функция выглядит так:

```
void structRead(FILE *file, Recipe *dishes, int num){
  for(int i = 0; i<num; i++)
  fread(&dishes[i], sizeof(Recipe), 1, file);
  fseek(file, 0, SEEK_SET);
}
```

Функция `structWrite()` используется для записи структур в файл. Как и для `structRead()`, `sctructWrite()` принимает как значения ссылку на файл, массив структур и количество записываемых структур. Реализация в коде выглядит так:

```
void structWrite(FILE *file, Recipe *dishes, int num){
  for(int i = 0; i < num; i++)
    fwrite(&dishes[i], sizeof(Recipe), 1, file);

  fseek(file, 0, SEEK_SET);
}
```

2 АЛГОРИТМЫ СОРТИРОВКИ

Алгоритм сортировки — это алгоритм для упорядочивания элементов в списке. В случае, когда элемент в списке имеет несколько полей, поле, служащее критерием порядка, называется ключом сортировки. На практике в качестве ключа часто выступает число, а в остальных полях хранятся какие-либо данные, никак не влияющие на работу алгоритма [2]. При помощи сортировок пользователь может систематизировать данные тем самым облегчая работу и доступ к информации.

2.1 Сортировка пузырьком

Сортировка пузырьком, по сравнению с другими алгоритмами считается простейшим для понимания и реализации. Эффективен для массивов небольшого размера.

Выполняется некоторое количество проходов по массиву — начиная от начала массива, перебираются пары соседних элементов массива. Если 1-й элемент пары больше 2-го, элементы переставляются.

Пары элементов массива перебираются либо раз, либо до тех пор, пока на очередном проходе не обнаружится, что более не требуется выполнять перестановки массив отсортирован.

При каждом проходе алгоритма по внутреннему циклу очередной наибольший элемент массива ставится на своё место в конце массива рядом с предыдущим «наибольшим элементом», а наименьший элемент перемещается на одну позицию к началу массива (как бы «всплывает» до нужной позиции, как пузырёк в воде — откуда и название алгоритма) [2].

В курсовой работе её код выглядит так:

```
void bubblesort(Recipe *dishes, int num){
for(int i = 0; i < num-1; i++){
     for(int j = 0; j < num-i-1; j++){
          if(strcmp(dishes[j].name, dishes[j+1].name)>0){
               Recipe temp = dishes[j];
               dishes[j] = dishes[j+1];
               dishes[j+1] = temp;
               }
          }
     }
}
```

Строка `for(int i = 0; i < num-1; i++)` необходима для того, чтобы перемещать самый большой элемент в конец массива.

Далее идёт вложенный цикл `for(int j = 0; j < num-i-1; j++)`, который в каждой итерации проходит по массиву и сравнивает элементы.

Следующие строки, в случае если первый символ выбранной строки больше, чем первый символ следующей, меняет их местами, путём создания

временной переменной, которой присваивается значение 1-го элемента, 1-му элементу присваивается значение 2-го, а 2-му значение 1-го:

```
        if(strcmp(dishes[j].name, dishes[j+1].name)>0){
             Recipe temp = dishes[j];
             dishes[j] = dishes[j+1];
             dishes[j+1] = temp;
             }
        }
```

2.2 Сортировка по категориям

Эта сортировка используется для вывода информации, расположены в алфавитном порядке и разделенной на категории. Для её реализации используется тот же алгоритм, что и для сортировки пузырьком, лишь с тем отличием, что в начале массив сортируется по номеру типа блюда, а именно typenum, где 1 — первое, 2 — второе, 3 — десерт и 4 — напиток:

```
void sort_for_print(Recipe *dishes, int num){
     for (int i = 0; i < num - 1; ++i) {
     for (int j = 0; j < num - i - 1; ++j) {
     if (dishes[j].typenum > dishes[j + 1].typenum) {
     Recipe temp = dishes[j];
     dishes[j] = dishes[j + 1];
     dishes[j + 1] = temp;
               }
          }
     }
     for(int i = 0; i<num-1; i++){
          for(int j = 0; j<num-i-1; j++){
          if((dishes[j].typenum ==
     dishes[j+1].typenum)&&(strcmp(dishes[j].name,
     dishes[j+1].name)>0)){
          Recipe temp = dishes[j];
          dishes[j] = dishes[j + 1];
          dishes[j + 1] = temp;
          break;
               }
          }
     }
}
```

Первые 2 цикла, как и в сортировке пузырьком перемещают больший элемент в конец массива проходятся по массиву и меняют местами больший и меньший элементы. Другие 2 делают то же самое, только меняют элементы в случае, если 1-ый символ выбранного элемента больше 1-го символа следующего и тип блюда выбранного элемента равен типу блюда следующего элемента.

2.3 Быстрая сортировка

Быстрая сортировка, сортировка Хоара — алгоритм сортировки, разработанный английским информатиком Тони Хоаром во время его работы в МГУ в 1960 году.

Один из самых быстрых известных универсальных алгоритмов сортировки массивов: в среднем обменов при упорядочении элементов; из-за наличия ряда недостатков на практике обычно используется с некоторыми доработками [2].

В коде эта сортировка разделена на 2 функции: `quicksort` и `partition`:

```
int partition(Recipe *dishes, int start, int end)
{
     int pivot = dishes[end].kall;
     int pIndex = start;
     for (int i = start; i < end; i++){
          if (dishes[i].kall <= pivot)
          {
               Recipe temp = dishes[i];
               dishes[i] = dishes[pIndex];
               dishes[pIndex] = temp;
               pIndex++;}
     }
     Recipe temp = dishes[pIndex];
     dishes[pIndex] = dishes[end];
     dishes[end] = temp;
     return pIndex;}
```

Функция `partition` принимает на вход массив структур `Recipe`, индексы 1-го (`start`) и последнего (`end`) элементов в целочисленном формате.

Переменная `pivot` является индексом опорного элемента, то есть элементом, с которым будут сравниватся остальные для дальнейшей сортировки.

Переменная `pIndex` содержит в себе индекс позиции, в которую будут помещаться меньшие элементы.

Цикл проходится по всем элементам, и в случае, если количество калории текущего элемента меньше или равен `pivot`, то меняет этот элемент местами с `pivot`-элементом.

Далее меняются местами элементы под индексами `end` и `pIndex`, после чего все элементы, которые меньше `pivot` — становятся слева, а большие — справа.

```
void quicksort(Recipe *dishes, int start, int end)
{
     if (start >= end) {
          Return;}
     int pivot = partition(dishes, start, end);
     quicksort(dishes, start, pivot - 1);
```

```
        quicksort(dishes, pivot + 1, end);}
```

Функция `quicksort` принимает на вход массив структур `Recipe`, индексы 1-го (`start`) и последнего (`end`) элементов в целочисленном формате `int`.

В начале сравниваются обе переменные, и если `start` больше или равен `end`, то выходим из функции.

В случае, если нет, вызывается функцию `partition`, и передаём ей массив структур `Recipe`, первый элемент и последний, и вывод присваевается функции переменной `pivot`, которая будет являться индексом опорного элемента.

Далее рекурсивно вызывается функция `quicksort` и передаём ей массив, начальный индекс `start` и конечный `pivot - 1` для сортировки рецептов перед `pivot` и также вызывается `quicksort` и передаём ей массив, начальный элемент `pivot + 1` и конечный `end`.

2.4 Сортировка выбором

Суть алгоритма сортировки выбором заключается в том, что на каждом шаге выбирается минимальный элемент из оставшейся части массива и перемещается в начало. Сложность алгоритма: $O(n^2)$[3].

В коде функции передаётся массив структур `Recipes`, и количество элементов массива.

```
void selectionSort(Recipe *dishes, int n) {
     for (int i = 0; i < n - 1; i++) {
          int minIndex = i;
          for (int j = i + 1; j < n; j++) {
               if (dishes[j].time < dishes[minIndex].time) {
                    minIndex = j;
               }
          }
          int temp = dishes[minIndex];
          dishes[minIndex] = dishes[i];
          dishes[i] = temp;
     }
}
```

Цикл проходится по массиву. На каждой итерации `i` проводится поиск наименьшего элемент в оставшейся части массива. В строке `int minIndex = i;` предполагается, что текущий элемент является минимальным.

Вложенный цикл проходит по неотсортированной части массива, и в случае, если поле времени `j`-ой структуры Recipe меньше, чем поле времени минимального элемента, то переменная `minIndex` обновляется на `j`.

После окончания работы вложенного цикла, меняются местами `i`-ый элемент с минимальным элементом. В последующих итерациях текущий элемент будет меняться до тех пор, пока не станет конечным, что означает, что массив отсортирован.

3 АЛГОРИТМЫ ПОИСКА

Поиск — обработка некоторого множества данных с целью выявления подмножества данных, соответствующего критериям поиска [4]. Существует огромное количество алгоритмов, но в этой работе будет рассмотрено только 2: линейный и интерполяционный.

3.1 Линейный поиск

Линейный, последовательный поиск — алгоритм нахождения заданного значения произвольной функции на некотором отрезке. Данный алгоритм является простейшим алгоритмом поиска и, в отличие, например, от двоичного поиска, не накладывает никаких ограничений на функцию и имеет простейшую реализацию. Поиск значения функции осуществляется простым сравнением очередного рассматриваемого значения и, если значения совпадают, то поиск считается завершённым [2].

```
void linearSearch(FILE* log, Recipe *dishes, int num){
     char name[100];
     cout << "Введите название рецепта: ";
     cin.ignore();
     cin.getline(name, 90);
     printTitle(log);
     for(int i = 0; i<num; i++){
          if(strcmp(dishes[i].name, name)==0){
          fprintf(stdout,"%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
          100s\n",dishes[i].name,dishes[i].type,dishes[i].ti
          me,dishes[i].kall,dishes[i].description);

          fprintf(file,"%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
          100s\n",dishes[i].name,
          dishes[i].type,dishes[i].time,dishes[i].kall,
          dishes[i].description);
          return;}
          }
     cout << "not found";}
```

Функции передаётся ссылка на файл `log` для лог-файла. Также передаётся массив структур `Recipe`, в котором будет проводиться поиск и целочисленная переменная типа `int`, обозначающая количество элементов.

В начале запрашивается название рецепта у пользователя и записывается в массив типа `char`.

Далее в цикле перебираются все элементы массива, и введённое пользователем название сравнивается с полем `name` каждого элемента. В случае, если они совпадают, выводится заголовок таблицы, информация о рецепте и функция завершается. Иначе — выводится сообщение, что файл не найден и происходит выход из функции.

3.2 Интерполяционный поиск

Интерполяционный поиск вместо сравнения каждого элемента с искомым, как при линейном поиске, производит предсказание местонахождения элемента: поиск происходит подобно двоичному поиску, но вместо деления области поиска на две части, интерполирующий поиск производит оценку новой области поиска по расстоянию между ключом и текущим значением элемента. Другими словами, бинарный поиск учитывает лишь знак разности между ключом и текущим значением, а интерполирующий ещё учитывает и модуль этой разности и по данному значению производит предсказание позиции следующего элемента для проверки. В среднем интерполирующий поиск производит log(log(N)) операций, где N есть число элементов, среди которых производится поиск. Число необходимых операций зависит от равномерности распределения значений среди элементов. В плохом случае (например, когда значения элементов экспоненциально возрастают) интерполяционный поиск может потребовать до O(N) операций [2].

Для корректной работы, массив, с которым будет работать функция, должен быть отсортирован перед поиском. Потому в коде перед её вызовом, массив сортируется алгоритмом быстрой сортировки.

```
fprintf(log, "\n### Interpolated search by calories ###\n");
quicksort(dishes, 0, num-1);
interpolationSearch(dishes, num, kal, log);
```

В начале функция запрашивает на вход целочисленные переменные типа `int x` и `n`, где `x` — искомое значение, n — количество элементов. Также функции необходимы ссылка на файл `log` для лог-файла и массив структур `Recipe`, в котором производится поиск:

```
void interpolationSearch(Recipe *dishes, int n, int x, FILE
*log)
```

Далее устанавливается нижний (`low`) и верхний (`high`) пределы массива и булевую переменную `problem` для отслеживания статуса сортировки.

Пока нижний предел меньше либо равен верхнему, `x` меньше поля `kall` элемента нижнего предела и больше верхнего предела, вычисляется примерная позиция искомого элемента.

```
int pos = low + ((x - dishes[low].kall) * (high - low)) /
(dishes[high].kall - dishes[low].kall);
```

Далее она сравнивается с искомым результатом. Если поле `kall` элемента равно искомому, то выводится результат и происходит выход из программы. Иначе в случае, если искомый элемент больше, нижнему пределу

присваивается значение `pos+1`, иначе – верхнему пределу присваивается `pos-1`.

```
if (dishes[pos].kall == x) {
     printTitle(log);
     fprintf(stdout, "%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
     100s\n", dishes[pos].name, dishes[pos].type,
     dishes[pos].time,
     dishes[pos].kall,dishes[pos].description);
     fprintf(log, "%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
     100s\n", dishes[pos].name, dishes[pos].type,
     dishes[pos].time, dishes[pos].kall,
     dishes[pos].description);
     problem = false;
}

if (dishes[pos].kall < x) {
     low = pos + 1;
}

else {
     high = pos - 1;
}
```

Если после окончания цикла элемент не был найден, значит его не существует, выводится сообщение что элемент не найден и производится выход из функции.

```
if(problem){
     printTitle(log);
     cout << "not found\n";
}
```

4 ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

Для исключения повторения кусков кода в программе создаются функции, которые не только помогают немного экономить вес файла, но и улучшают и облегчают читаемость кода. В этой главе будут рассмотрены функции, необходимые для работы программы.

4.1 Вспомогательные функции

Функция `openNewFile` принимает на вход ссылку на файл `file` и массив символов `fileName`.

```
void openNewFile(FILE *file, char *fileName){
     if(IsExistsFile(file, fileName)){
          cout << "file " << fileName << " was created\n";
          create(file, fileName);
     }
}
```

Она, в случае если файла не существует, выводит сообщение что файл с заданным именем создан, и создаёт его.

Функция `checkIsEmpty` проверяет пустой ли файл.

```
int chekIsEmpty(FILE* file, const char filename[100]){
     file = fopen(filename, "r");
     int num = countStructs(file);
     fclose(file);

     return num>0? 0: 1;
}
```

В переменную `num` записывается количество элементов в файле. В случае, если он пуст, функция передаёт 1, иначе — 0.

Функция `printTitle` выводит заголовок для таблицы, который предназначен для подписи информации при выводе рецептов.

```
void printTitle(FILE *file){
     fprintf(stdout, "%-15s%-20s%-20s%-20s%-100s\n",
     "Name:", "type", "time", "calories", "description");

     fprintf(file, "%-15s%-20s%-20s%-20s%-100s\n", "Name:",
     "type", "time", "calories", "description");
}
```

Функция `printMassage` предназначена для вывода пунктов меню.

```
void printMessage(){
```

```
        cout << "\n 1 - добавить рецепт(ы).\n 2 -
        редактирование записей в файле.\n 3 - Удаление записей
        из файла.\n\n "
        "4 - сортировки: \n\t4.1 - сортировка и категоризация
        \n\t4.2 - сортировка пузырьком \n\t4.3 - быстрая
        сортировка \n\t4.4 - сортировка выбором\n\t4.5 -
        выход\n\n"
        "5 - поиски: \n\t5.1 - линейный \n\t5.2 -
        интерполяционный\n\t5.3 - выход\n\n"
        "6 - поиск диетического рецепта.\n"
        "7 - вывод меню\n"
        "8 - выйти из программы\n"
        "9 - вывести меню\n";
}
```

4.2 Функции для работы с рецептами

Прежде всего, в программе существует функция добавления рецептов `add`, которая принимает на вход ссылки на файлы `log` для лог-файлов и `file` для бинарного файла с рецептами, и массив символов `fileName` для открытия определённого файла.

В самой функции, в случае если при создании файла не возникает ошибка, пользователя просят ввести количество рецептов, которое будет добавлено. После этого начинает работать цикл, который введённое количество раз будет запрашивать пользователя информацию об рецептах и записывать их в бинарный файл для дальнейшего хранения.

Функция `delEl` принимает на вход ссылки на файлы `log` для лог-файлов и `file` для бинарного файла с рецептами, и массив символов `fileName` для открытия определённого файла.

```
bool delele = false;
     for(int i = 0; i<num; i++){
     if(strcmp(dishes[i].name, name)==0){
          num--;
          dishes[i] = dishes[i+1];
          delele = true;
     }
     if(delele)
          dishes[i]=dishes[i+1];
     }
     if(delele){
          cout << "element is deleted\n";
          fprintf(log, "element is deleted\n");
     } else {
          cout << "something went wrong...\n";
          fprintf(log, "something went wrong...\n");
     }
structWrite(file, dishes, num);
```

Функция считывает массив структур `Recipe` из бинарного файла, просит пользователя ввести название рецепта, и в случае, если в последующем цикле был найден элемент, он будет удалён из массива, а после этого будет записан снова в бинарный файл.

Функция `edit` принимает на вход ссылки на файлы `log` для лог-файлов и `file` для бинарного файла с рецептами, и массив символов `fileName` для открытия определённого файла.

Её алгоритм схож на функцию `delEl`. Она считывает структуры из бинарного файла в массив. Пользователя просят ввести название рецепта, который будут редактировать. После этого в массиве производится поиск элемента по названию. Если он находится, то пользователя просят ввети новые данные для рецепта.

```
structWrite(file, dishes, num);
cout << "element was edited\n";
fprintf(log, "\n### Element %20s was edited ###\n", name);
fclose(file);
delete [] dishes;
```

Функция `dietprint` принимает на вход ссылки на файлы `log` для лог-файлов и `file` для бинарного файла с рецептами, и массив символов `fileName` для открытия определённого файла.

Функция считывает структуры из бинарного файла в массив и считает количество их.

```
log = fopen("log.txt", "a");
file = fopen(fileName, "rb");
// openFile(file, fileName, "rb");
int num = countStructs(file);
Recipe *dishes = new Recipe[num];
structRead(file, dishes, num);
fclose(file);
```

Далее она запрашивает пользователя калории и вызывает функцию selecti. В последующем цикле введённые калорпии сравниваются с полем kall всех элементов. В случае, если поле меньше или равно, то выводится информация о рецепте.

```
cout << "Enter calories: ";
cin >> kal;
selectionSort(dishes, num);
for(int i = 0; i < num; i++) {
     if(dishes[i].kall < kal){
          fprintf(stdout,"%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
          100s\n",dishes[i].name,dishes[i].type,dishes[i].ti
          me,dishes[i].kall,dishes[i].description);
```

```
            fprintf(log,"%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
            100s\n",dishes[i].name,dishes[i].type,dishes[i].ti
            me,dishes[i].kall,dishes[i].description);
        }
    }
```

Функция `printop` принимает на вход ссылки на файлы `log` для лог-файлов и `file` для бинарного файла с рецептами, массив символов `fileName` для открытия определённого файла, и целочисленную переменную типа `int` для номера операции.

Функция считывает структуры из бинарного файла в массив и считает количество их, а также создаёт булевую переменную `isSort` для того, чтобы далее правильного вывода в консоль и лог-файл.

Далее, в зависимости от переданного номера операции, будет вызвана функция сортировки или поиска. Если будет вызвана функция сортировки, то `isSort` станет равным `true`. Если `isSort` равен `true`, то выводится заголовок таблицы и массив рецептов.

```
    if(isSort){
        printTitle(log);
        printForSort(log, num, dishes);
    }
```

5 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

При запуске программы пользователя просят ввести имя файла, с которым он желает работать:

```
Enter filename: recipes
file recipes was created
```

Рисунок 5.1

Далее в консоли появляется меню, которое отображает все возможные операции с программой, и пользователя просят ввести номер операции:

```
 1 - добавить рецепт(ы).
 2 - редактирование записей в файле.
 3 - Удаление записей из файла.

 4 - сортировки:
        4.1 - сортировка и категоризация
        4.2 - сортировка пузырьком
        4.3 - быстрая сортировка
        4.4 - сортировка выбором
        4.5 - выход

5 - поиски:
        5.1 - линейный
        5.2 - интерполяционный
        5.3 - выход

6 - поиск диетического рецепта.
7 - вывод меню
8 - выйти из программы
enter number of item: 
```

Рисунок 5.2

При вызове всех функций для пустого файла (кроме добавления, вывода меню и выхода из программы) будет выведено сообщение, что файл пуст:

```
enter number of item: 2
file is empty
```

Рисунок 5.3

В начале пользователя просят ввести количество добавляемых рецептов. Эта функция была добавлена для удобства ввода некоторого количества рецептов за раз:

```
enter number of item: 1
Enter number of recipes: 1
Enter name of recipe: Кето "жареная картошка"
Enter type of recipe (1 - first, 2 - second, 3 - dessert, 4 - drink): 1
Enter calories: 294
Enter time: 15
Enter description: Отличное полноценное блюдо для всей семьи
element was added
```

Рисунок 5.1

В случае, если пользователь посчитает нужным исключить из списка какой-либо из рецептов, он может выбрать пункт меню “удаление записей из списка”. Тогда пользователю нужно будет ввести название рецепта, после чего возникнет сообщение об успешности операции:

```
Enter recipe name: сырники
element is deleted
```

Рисунок 5.1

Или программа может вывести ошибку, если не сможет отыскать элеиент, который ввёл пользователь:

```
Enter recipe name: драники
something went wrong...
```

Рисунок 5.1

Ввод для сортировок требует всего лишь выбора типа сортировок. Так, например, для сортировки и категоризации, необходимо выбрать лишь сам пункт в меню, после чего будет выведен отсортированный список в табличном виде:

```
enter number of item: 4
enter number of sub-item: 1
Name:                    type                    time                    calories                    description

Кето "жареная картошка"          |      first          |15          |  294          |   Отличное полноценное блюдо для всей семьи

свинина в кунжуте                |  second             |20          |  180          |   простой перекус

сырники                          |  dessert            |30          |  245          |   Вкусный и простой рецепт

белковый коктейль с бананом      |  drink              |3           |  71           |   вкусный напиток для большой компании
```

Рисунок 5.1

При поиске необходимо выбрать не только тип поиска, но и то, что просит программа.

В случае если программа находит элемент, она выводит его следующим образом:

```
Name:          type                    time                    calories                    description

сырники |      dessert                 |30                  |  245       |     Вкусный и простой рецепт
```

Рисунок 5.1

Если же нет, то так:

```
Name:          type                    time                    calories                description

not found
```

Рисунок 5.1

При поиске диетического рецепта пользователя просят ввести калории, выше которых рецепты просто не будут выводиться, то есть количество калорий всех выведенных рецептов не будет превышать тех калорий, которых ввёл пользователь:

```
enter number of item: 6
Enter calories: 240
белковый коктейль с бананом|    drink                    |3                  |  71        |     вкусный напиток для большой компании
свинина в кунжуте|      second                  |20                  |  180       |     простой перекус
```

Рисунок 5.1

После окончания работы с программой, можно завершить её работу, после чего можно, в случае необходимости, снова запустить программу с целью редактирования другого файла.

ЗАКЛЮЧЕНИЕ

В данной работе была успешно разработана программа по управлению электронными рецептами, позволяющая автоматизировать рутинную работу. Актуальность данной работы подтверждается тем, что существует не очень много подобных приложений, позволяющих также облегчить работу с электронными рецептами, при этом также позволяющие работать с несколькими файлами параллельно.

Разработано программное обеспечение на языке программирования C/C++ в среде XCode. В его основе лежат несколько алгоритмов сортировок и поиска, обеспечивающие быструю обработку информации о рецептах. В программе также был добавлен интерфейс для удобного управления командами, а также имеет возможность создания и просмотра лог-файла для просмотра истории изменения книги рецептов.

Также не маловажным является то, что этот код улучшает производительность 1 человека, путём автоматизации рутинной работы, что может обеспечить экономию временных и трудовых ресурсов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Metanit.com [Электронный ресурс] - Режим доступа: https://metanit.com/cpp/tutorial/

[2] Wikipedia.org [Электронный ресурс] - Режим доступа: https://ru.wikipedia.org/

[3] Habr [Электронный ресурс] - Режим доступа: https://habr.com/ru/

[4] prog-cpp.com [Электронный ресурс] - Режим доступа: https://prog-cpp.ru/algorithm-sort/

[5] Б. Керниган, Д. Ритчи. Язык программирования Си. 2-е изд: «Невский Диалект», 2003. – 355с.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода

```
#include <iostream>
#include <cstdio>
#include <limits>
#include <cstring>

using namespace std;

struct Recipe{
     // 1 - первое 2 - второе 3 - десерт 4 - напиток
     char type[18];
     int typenum;
     char name[100];
     int kall;
     int time;
     char description[200];
};

int countStructs(FILE *file){
     fseek(file, 0, SEEK_END);
     int num = (int)(ftell(file)/sizeof(Recipe));
     fseek(file, 0, SEEK_SET);
     return num;
}

void structRead(FILE *file, Recipe *dishes, int num){
     for(int i = 0; i<num; i++)
     fread(&dishes[i], sizeof(Recipe), 1, file);
     fseek(file, 0, SEEK_SET);
}

void structWrite(FILE *file, Recipe *dishes, int num){
     for(int i = 0; i < num; i++)
     fwrite(&dishes[i], sizeof(Recipe), 1, file);

     fseek(file, 0, SEEK_SET);
}

void printTitle(FILE *file){
     fprintf(stdout, "%-15s\t%-20s\t%-20s\t%-20s\t%-100s\n",
     "Name:", "type", "time", "calories", "description");
     fprintf(file, "%-15s\t%-20s\t%-20s\t%-20s\t%-100s\n",
     "Name:", "type", "time", "calories", "description");
}

int IsExistsFile(FILE* file, char fileName[100]){
     file = fopen(fileName, "rb");
     int check = !file? 1: 0;
     fclose(file);
     return check;
```

```
}
```

Продолжение приложения А

```
void create(FILE *file, char fileName[100]){
     file = fopen(fileName, "wb");

     if(!file){
          cout << "Create file error!\n";
          return;
     }
     fclose(file);

}

void openNewFile(FILE *file, char *fileName){
     if(IsExistsFile(file, fileName)){
          cout << "file " << fileName << " was created\n";
          create(file, fileName);
     }
}

int chekIsEmpty(FILE* file, const char filename[100]){
     file = fopen(filename, "r");
     int num = countStructs(file);
     fclose(file);

     return num>0? 0: 1;
}

void add(FILE* log, FILE *file, char fileName[100]){
     Recipe dish;
     int n;
     int iche;
     bool problem = true;
     file = fopen(fileName, "ab");
     log = fopen("log.txt", "a");
     if(!file || !log){
          cout << "Open file error!\n";
          return;
     }
     cout << "Enter number of recipes: ";
     cin >> n;
     cin.ignore(numeric_limits<streamsize>::max(),'\n');

     for(int i = 0; i < n; i++){
          cout << "Enter name of recipe: ";
          cin.getline(dish.name, 90);
          cout << "Enter type of recipe (1 - first, 2 - second, 3
          - dessert, 4 - drink): ";
          problem = true;
          cin >> iche;
          while(problem){
               switch (iche) {
               case 1:
               strcpy(dish.type, "first");
```

```
                dish.typenum = 1;
                problem = false;
```

Продолжение приложения А

```
                break;
                case 2:
                strcpy(dish.type, "second");
                dish.typenum = 2;
                problem = false;
                break;
                case 3:

                strcpy(dish.type, "dessert");
                dish.typenum = 3;
                problem = false;
                break;
                case 4:
                strcpy(dish.type, "drink");
                problem = false;
                dish.typenum = 4;
                break;
                default:
                cout << "no\n";
                cin >> iche;
                break;
                }
            }
            cout << "Enter calories: ";
            while(true){
                cin >> dish.kall;
                if(dish.kall>=0){
                    break;
                }
            }
            cout << "Enter time: ";
            while(true){
                cin >> dish.time;
                if(dish.time>=0){
                    break;
                }
            }
            cout << "Enter description: ";
            cin.ignore(numeric_limits<streamsize>::max(),'\n');
            cin.getline(dish.description, 190);
            fwrite(&dish, sizeof(Recipe), 1, file);
        }
        cout << "element was added\n";
        fprintf(log, "\n### Element %20s was added ###\n",
        dish.name);
        fclose(file);
    }

    void sort_for_print(Recipe *dishes, int num){
        for (int i = 0; i < num - 1; ++i) {
            for (int j = 0; j < num - i - 1; ++j) {
                if (dishes[j].typenum > dishes[j + 1].typenum) {
```

```
                    Recipe temp = dishes[j];
                    dishes[j] = dishes[j + 1];
                    dishes[j + 1] = temp;
```

Продолжение приложения А

```
                    }
               }
               }
               for(int i = 0; i<num-1; i++){
                    for(int j = 0; j<num-i-1; j++){
                         if((dishes[j].typenum ==
                         dishes[j+1].typenum)&&(strcmp(dishes[j].
                         name, dishes[j+1].name)>0)){
                         Recipe temp = dishes[j];
                         dishes[j] = dishes[j + 1];

                         dishes[j + 1] = temp;
                         break;
               }
          }
     }
}

void bubblesort(Recipe *dishes, int num){
     for(int i = 0; i < num-1; i++){
          for(int j = 0; j < num-i-1; j++){
               if(strcmp(dishes[j].name, dishes[j+1].name)>0){
               Recipe temp = dishes[j];
               dishes[j] = dishes[j+1];
               dishes[j+1] = temp;
               }
          }
     }
}

int partition(Recipe *dishes, int start, int end)
{
     // Выбираем крайний правый элемент в качестве опорного
     элемента массива
     int pivot = dishes[end].kall;
     int pIndex = start;
     for (int i = start; i < end; i++)
     {
               if (dishes[i].kall <= pivot)
               {
                    Recipe temp = dishes[i];
                    dishes[i] = dishes[pIndex];
                    dishes[pIndex] = temp;
                    pIndex++;
               }
          }
          Recipe temp = dishes[pIndex];
          dishes[pIndex] = dishes[end];
          dishes[end] = temp;
          return pIndex;
     }
```

```
void quicksort(Recipe *dishes, int start, int end)
{
```

Продолжение приложения А

```
     if (start >= end) {
     return;
     }

     int pivot = partition(dishes, start, end);

     quicksort(dishes, start, pivot - 1);

     quicksort(dishes, pivot + 1, end);
}

void selectionSort(Recipe *dishes, int n) {
     for (int i = 0; i < n - 1; i++) {
          int minIndex = i;
          for (int j = i + 1; j < n; j++) {

               if (dishes[j].time < dishes[minIndex].time) {
                    minIndex = j;
          }
     }
     // Обмениваем текущий элемент и минимальный элемент
     Recipe temp = dishes[minIndex];
     dishes[minIndex] = dishes[i];
     dishes[i] = temp;
     }
}

void delEl(FILE* log, FILE *file, char fileName[100]){
     char name[100];

     file = fopen(fileName, "rb");
     log = fopen("log.txt", "w");
     if(!file || !log){
          cout << "Open file error!\n";
          return;
     }
     int num = countStructs(file);
     Recipe *dishes = new Recipe[num];
     structRead(file, dishes, num);
     fclose(file);


     file = fopen(fileName, "wb");
     cout << "Enter recipe name: ";
     cin.ignore(numeric_limits<streamsize>::max(), '\n');
     cin.getline(name, 100);
     bool delele = false;
     for(int i = 0; i<num; i++){
          if(strcmp(dishes[i].name, name)==0){
               num--;
          dishes[i] = dishes[i+1];
```

```
            delele = true;
        }
        if(delele)
            dishes[i]=dishes[i+1];
```

Продолжение приложения А

```
        }
        if(delele){
            cout << "element is deleted\n";
            fprintf(log, "element is deleted\n");
        } else {
            cout << "something went wrong...\n";
            fprintf(log, "something went wrong...\n");
        }
        structWrite(file, dishes, num);
        fclose(file);
        fprintf(log, "\n### Element %20s was deleted ###\n", name);
        fclose(log);
        delete [] dishes;
}

void edit(FILE* log, FILE *file, char fileName[100]){
        char name[100];

        file = fopen(fileName, "rb");

        log = fopen("log.txt", "a");
        if(!file || !log){
            cout << "Open file error!\n";
            return;
        }
        int num = countStructs(file);
        Recipe *dishes = new Recipe[num];
        structRead(file, dishes, num);
        fclose(file);

        // openFile(file, fileName, "wb");
        file = fopen(fileName, "wb");
        cout << "Enter recipe name: ";
        cin.ignore(numeric_limits<streamsize>::max(),'\n');
        cin.getline(name, 100);
        int iche;
        for(int i = 0; i<num; i++){
            if(strcmp(dishes[i].name, name)==0){
                cout << "Enter name: ";
                cin.getline(dishes[i].name, 100);
                cout << "Enter type of recipe (1 - first, 2 -
                second, 3 - dessert, 4 - drink): ";
                cin >> iche;
                bool problem = true;
                while(problem){
                    switch (iche) {
                    case 1:
                        strcpy(dishes[i].type, "first");
                        dishes[i].typenum = 1;
                        problem = false;
```

```
                    break;
                case 2:
                    strcpy(dishes[i].type, "second");
                    dishes[i].typenum = 2;
                    problem = false;
```

Продолжение приложения А

```
                    break;
                case 3:
                    strcpy(dishes[i].type, "dessert");
                    dishes[i].typenum = 3;
                    problem = false;
                    break;
                case 4:
                    strcpy(dishes[i].type, "drink");
                    problem = false;
                    dishes[i].typenum = 4;
                    break;
                default:
                    cout << "no\n";
                    cin >> iche;
                    break;
                }
            }
            cout << "Enter calories: ";
            while(true){
                cin >> dishes[i].kall;
                if(dishes[i].kall>=0){
                    break;
                }

            }
            cout << "Enter time: ";
            while(true){
                cin >> dishes[i].time;
                if(dishes[i].time>=0){
                    break;
                }
            }
            cout << "Enter description: ";
            cin.ignore(numeric_limits<streamsize>::max(),
            '\n');
            cin.getline(dishes[i].description, 190);
        }
    }
    structWrite(file, dishes, num);
    cout << "element was edited\n";
    fprintf(log, "\n### Element %20s was edited ###\n", name);
    fclose(file);
    delete [] dishes;
}

void interpolationSearch(Recipe *dishes, int n, int x, FILE
*log) {
    int low = 0, high = n - 1;
    bool problem = true;
```

```
	while (low <= high && x >= dishes[low].kall && x <=
	dishes[high].kall) {

		int pos = low + ((x - dishes[low].kall) * (high - low))
		/ (dishes[high].kall - dishes[low].kall);
```

Продолжение приложения А

```
		if (dishes[pos].kall == x) {
			printTitle(log);
			fprintf(stdout, "%-15s|\t%-20s\t|%-20d|\t%-
			10d|\t%-100s\n", dishes[pos].name,
			dishes[pos].type, dishes[pos].time,
			dishes[pos].kall,dishes[pos].description);
			fprintf(log, "%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
			100s\n", dishes[pos].name, dishes[pos].type,
			dishes[pos].time, dishes[pos].kall,
			dishes[pos].description);
			problem = false;
		}

		if (dishes[pos].kall < x) {
			low = pos + 1;
		}

		else {
			high = pos - 1;
		}
		}
		if(problem){
			printTitle(log);
			cout << "not found\n";
		}
}

void dietprint(FILE* log, FILE *file, char fileName[100]){
	int kal;

	// openFile(log, "log.txt", "w");
	log = fopen("log.txt", "a");
	file = fopen(fileName, "rb");
	// openFile(file, fileName, "rb");
	int num = countStructs(file);
	Recipe *dishes = new Recipe[num];
	structRead(file, dishes, num);

	fclose(file);

	cout << "Enter calories: ";
	cin >> kal;
	selectionSort(dishes, num);
	for(int i = 0; i < num; i++) {
		if(dishes[i].kall < kal){
			fprintf(stdout, "%-15s|\t%-20s\t|%-20d|\t%-
			10d|\t%-100s\n", dishes[i].name, dishes[i].type,
```

```
                dishes[i].time, dishes[i].kall,
                dishes[i].description);
                fprintf(log, "%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
                100s\n",
```

Продолжение приложения А

```
                dishes[i].name, dishes[i].type, dishes[i].time,
                dishes[i].kall, dishes[i].description);
            }
        }
        fclose(log);
        delete [] dishes;
}

void linearSearch(FILE* log, Recipe *dishes, int num){
        char name[100];

        cout << "Введите название рецепта: ";
        cin.ignore();
        cin.getline(name, 90);

        printTitle(log);
        for(int i = 0; i<num; i++){
            if(strcmp(dishes[i].name, name)==0){
                fprintf(stdout, "%-15s|\t%-20s\t|%-20d|\t%-
                10d|\t%-100s\n", dishes[i].name, dishes[i].type,
                dishes[i].time,
                dishes[i].kall,dishes[i].description);
                fprintf(log, "%-15s|\t%-20s\t|%-20d|\t%-10d|\t%-
                100s\n", dishes[i].name, dishes[i].type,
                dishes[i].time, dishes[i].kall,
                dishes[i].description);
                return;
            }
        }
        cout << "not found\n";
}

void printForSort(FILE *file, int num, Recipe *dishes){
        for(int i = 0; i < num; i++) {
            fprintf(stdout, "%-60s|\t%-20s\t|%-20d|\t%-20d|\t%-
            100s\n", dishes[i].name, dishes[i].type,
            dishes[i].time, dishes[i].kall, dishes[i].description);
            fprintf(file, "%-60s|\t%-20s\t|%-20d|\t%-20d|\t%-
            100s\n", dishes[i].name, dishes[i].type,
            dishes[i].time, dishes[i].kall, dishes[i].description);
        }
}

void printMessage(){
        cout << "\n 1 - добавить рецепт(ы).\n 2 - редактирование
        записей в файле.\n 3 - Удаление записей из файла.\n\n "
        "4 - сортировки: \n\t4.1 - сортировка и категоризация
        \n\t4.2 - сортировка пузырьком \n\t4.3 - быстрая сортировка
        \n\t4.4 - сортировка выбором\n\t4.5 - выход\n\n"
```

```
     "5 - поиски: \n\t5.1 - линейный \n\t5.2 -
     интерполяционный\n\t5.3 - выход\n\n"
     "6 - поиск диетического рецепта.\n"
     "7 - вывод меню\n"
```

Продолжение приложения А

```
     "8 - выйти из программы\n";
}

void printop(FILE* log, FILE* file, char fileName[100], int
WhatToDo){
     bool isSort = false;

     // openFile(file, fileName, "rb");
     file = fopen(fileName, "rb");
     // openFile(log, "log.txt", "w");
     log = fopen("log.txt", "a");

     int num = countStructs(file);
     Recipe *dishes = new Recipe[num];
     structRead(file, dishes, num);

     fclose(file);

     switch (WhatToDo)
     {
          case 4:
               fprintf(log, "\n### Sort by category ###\n");
               sort_for_print(dishes, num);
               isSort = true;
               break;

          case 5:
               fprintf(log, "\n### Bubble sort by name ###\n");
               bubblesort(dishes, num);
               isSort = true;
               break;

          case 6:
          fprintf(log, "\n### quick sort by calories ###\n");

               quicksort(dishes, 0, num-1);
               isSort = true;
               break;

          case 7:
               fprintf(log, "\n### Selection sort by time
               ###\n");
               selectionSort(dishes, num);
               isSort = true;
               break;

          case 8:
               fprintf(log, "\n### linear search by name ###\n");
               linearSearch(log, dishes, num);
               break;
```

```
            case 9:
                int kal;
```

Продолжение приложения А

```
                cout << "Enter calories of recept: ";
                while (true){
                cin >> kal;
                if(kal>=0){
                break;
            }
            cout << "calories can't be less than 0!\n";
            }
            fprintf(log, "\n### Interpolated search by calories
            ###\n");
            quicksort(dishes, 0, num-1);
            interpolationSearch(dishes, num, kal, log);
        }

        if(isSort){
            printTitle(log);
            printForSort(log, num, dishes);
        }
        fclose(log);
        delete [] dishes;
    }

    int main() {
        int operation;
        int WhatToDo;
        bool isWork = true;
        FILE* file = nullptr;
        FILE* log = nullptr;
        char fileName[100];

        cout << "Enter filename: ";
        cin.getline(fileName, 100);

        openNewFile(file, fileName);
        printMessage();

        while(isWork){
            cout << "enter number of item: ";

            cin >> operation;
            switch (operation) {
                case 1:
                    add(log, file, fileName);
                    break;

                case 2:
                    if(chekIsEmpty(file, fileName)){
                        cout << "file is empty\n";
                        break;
                    }
```

```
     edit(log, file, fileName);
     break;

case 3:
     if(chekIsEmpty(file, fileName)){
```

Продолжение приложения А

```
          cout << "file is empty\n";
          break;
     }
     delEl(log, file, fileName);
     break;

case 4:
     if(chekIsEmpty(file, fileName)){
          cout << "file is empty\n";
          break;
     }
     cout << "enter number of sub-item: ";
     cin >> WhatToDo;
     switch(WhatToDo){
          case 1:
          //sort for print
          printop(log, file, fileName, 4);
          break;

          case 2:
          //bubble sort
          printop(log, file, fileName, 5);
          break;
          case 3:
          //quicksort
          printop(log, file, fileName, 6);
          break;

          case 4:
          //selection sort
          printop(log, file, fileName, 7);
          break;

          case 5:
          // выход
          break;
          default:
          printMessage();
          break;
     }
     break;
case 5:

if(chekIsEmpty(file, fileName)){
     cout << "file is empty\n";
     break;
}
cout << "enter number of sub-item: ";
cin >> WhatToDo;
```

```
                switch (WhatToDo){
                     case 1:
                     //linear search
                     printop(log, file, fileName, 8);
                     break;
```

Продолжение приложения А

```
                     case 2:
                     //interpolated sort
                     printop(log, file, fileName, 9);
                     break;

                     case 3:
                     // выход
                     break;
                     default:
                     printMessage();
                     break;
                }
                break;

           case 6:
                if(chekIsEmpty(file, fileName)){
                     cout << "file is empty\n";
                     break;
                }
                dietprint(log, file, fileName);
                break;
           case 7:
                printMessage();
                break;

           case 8:
                isWork = false;
                break;

           default:
                printMessage();
                break;
           }
      }
      return 0;
}
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Блок-схема работы программы

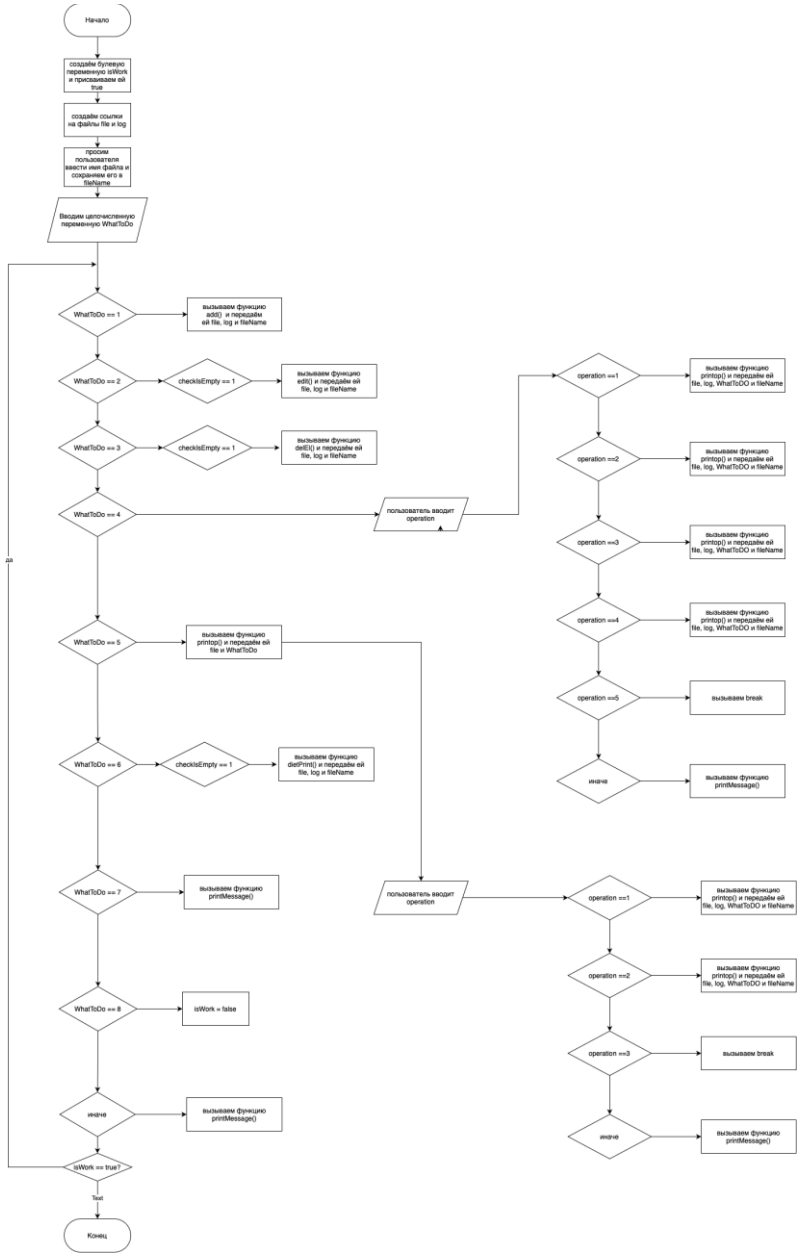

Рисунок Б.1 – Блок-схема работы программы

ВЕДОМОСТЬ ДОКУМЕНТОВ

Обозначение	Наименование	Дополнительные сведения
	Текстовые документы	
БГУИР КП 6-05-0611-03 065 ПЗ	Пояснительная записка	42 с.

Изм.	Л.	№ докум.	Подп.	Дата
Разраб.		Рассохов		
Проверил		Новицкая		

БГУИР КП 7-06-0611-03 065 ВД

Система организации электронных кулинарных рецептов

Лит	Лист	Листов
У	42	42

Кафедра ВМиП
гр. 421703